

EDV21C-4 Modbus RTU 协议说明

1. 适用范围

本文档定义 EDV21C-4 板型在当前项目中的 RS485 / Modbus RTU 协议。

- **物理层**: RS485 半双工 / UART 直连
- **协议层**: Modbus RTU 从机
- **角色关系**: 上位机 / PLC / 网关为主站, EDV21C-4 为从站

EDV21C-4 不会主动发送 Modbus RTU 帧, 主站必须轮询读取结果。

1.1 接口选择

当前固件下, EDV21C-4 对外使用统一的 Modbus RTU 协议, 物理接法通过拨码开关 1 选择:

拨码开关 1	通信方式	说明
ON	RS485	通过 A/B 端子接 Modbus RTU 主站
OFF	UART 直连	通过 UART0 直接发送 / 接收 Modbus RTU 帧

补充说明:

1. 两种模式下 **寄存器地址、功能码、帧格式完全一致**
2. 差别只在于物理层: ON 时使用 RS485 收发器, OFF 时使用 TTL UART 直连
3. UART 直连测试时, 发送的仍然是**完整 Modbus RTU 帧**, 不是 AT 指令, 也不是单独的雷达私有协议

1.2 A / B 端子与 UART 信号对应关系

当拨码开关 1 = OFF, 设备处于 UART 直连模式时, A / B 端子与 UART 信号的对应关系如下:

端子	设备信号方向	UART 含义	与外部系统连接关系
A	设备发送	Device TX	连接外部系统 RX
B	设备接收	Device RX	连接外部系统 TX

补充说明:

1. UART 直连模式下, 除 A / B 两根信号线外, 还应与外部系统共地
2. RS485 模式下, A / B 端子用于差分总线连接, 不再按 TX / RX 单端方向理解

2. 串口参数

项目	固定值
UART	UART0
引脚	TX=I021, RX=I020, DE=I02
波特率	9600
数据位	8
校验位	None
停止位	1
CRC	Modbus CRC16

补充说明:

1. 波特率固定为 9600 8N1, **不通过 Modbus 修改**
2. 从站地址仍由 BLE / 产测流程配置, **不在本文寄存器表中开放读写**
3. 由于 UART0 与下载 / 日志复用, 上电初期仍可能看到启动日志; 应用启动后切换到 RS485 工作
4. 当拨码开关 1 = OFF 时, 可直接把该组串口参数用于 UART 直连测试

3. 协议设计原则

3.1 不提供全局人数 / 全局目标总数

雷达当前结果是按 **感应区 zone_id** 分组输出的，并且不同感应区允许重叠。

因此同一个人可能同时出现在多个感应区结果中，所以协议 **不定义**：

- 全局人数
- 全局目标数量
- 全局目标列表

3.2 总体检测边界与分区配置分开

Modbus 对外模型拆成三层：

1. **全局参数**：灵敏度、目标消失延时
2. **总体检测边界**：安装高度、检测范围 XY、检测高度
3. **分区配置 / 分区结果**：感应区、屏蔽区、每个感应区的目标结果块

3.3 配置采用“暂存 + 应用”

主站写入 Holding Registers 时，先写入设备侧暂存镜像；只有在写 `apply_command` 后，设备才会把暂存值真正下发到运行配置和雷达模块。

4. 支持的 Modbus 功能码

功能码	说明
0x03	读 Holding Registers
0x04	读 Input Registers
0x06	写单个 Holding Register
0x10	写多个 Holding Registers

说明：

1. 0x06 适合写单个全局参数、总体检测边界参数或 `apply_command`
2. 0x10 适合一次写入完整的感应区 / 屏蔽区配置块
3. 不使用 Coil，因此 **不支持** 0x05 / 0x0F

5. Holding Registers

5.1 总览

地址	名称	类型	R/W	说明
0x0000	protocol_version	uint16	R	当前固定 0x0200
0x0001	motion_sensitivity	uint16	R/W	全局运动灵敏度，范围 0~10
0x0002	presence_sensitivity	uint16	R/W	全局存在灵敏度，范围 0~10
0x0003	target_disappear_delay	uint16	R/W	全局目标消失延时，单位 s，范围 >=2
0x0004	install_height_mm	uint16	R/W	雷达安装高度，单位 mm
0x0005	detect_x_min_mm	int16	R/W	总体检测范围 X 最小值，单位 mm
0x0006	detect_x_max_mm	int16	R/W	总体检测范围 X 最大值，单位 mm
0x0007	detect_y_min_mm	int16	R/W	总体检测范围 Y 最小值，单位 mm

地址	名称	类型	R/W	说明
0x0008	detect_y_max_mm	int16	R/W	总体检测范围 Y 最大值, 单位 mm
0x0009	detect_z_min_mm	uint16	R/W	总体检测高度最小值, 单位 mm
0x000A	detect_z_max_mm	uint16	R/W	总体检测高度最大值, 单位 mm
0x000B	sensing_zone_count	uint16	R/W	感应区数量, 范围 1~10
0x000C	mask_zone_count	uint16	R/W	屏蔽区数量, 范围 0~10
0x00F0	apply_command	uint16	W / R	写入应用命令; 读出最近一次命令值
0x00F1	apply_result	uint16	R	最近一次应用结果

说明:

1. 0x03 读到的是 Modbus **当前暂存镜像**
2. apply_command 成功执行后, 会把对应配置立即持久化到 NVS; **不需要额外的保存寄存器**

5.2 总体检测边界的含义

这一组寄存器对应 BLE / 雷达里的基础检测包络参数:

1. install_height_mm 对应安装高度
2. detect_x_min_mm ~ detect_y_max_mm 对应总体检测范围 XY
3. detect_z_min_mm ~ detect_z_max_mm 对应总体检测高度

说明:

1. 这组参数**不是**感应区, 也**不是**屏蔽区
2. 它们可以理解为感应区 / 屏蔽区可用的基础检测边界
3. 感应区和屏蔽区仍然是独立的业务分区配置

5.3 感应区定义块

- 起始地址: 0x0010
- 每个感应区占 7 个寄存器
- 最多 10 个感应区

每个感应区结构如下:

偏移	字段	类型	说明
+0	zone_id	uint16	范围 1~10
+1	x_min_mm	int16	单位 mm
+2	x_max_mm	int16	单位 mm
+3	y_min_mm	int16	单位 mm
+4	y_max_mm	int16	单位 mm
+5	z_min_mm	uint16	单位 mm
+6	z_max_mm	uint16	单位 mm

地址展开示例:

感应区槽位	地址范围
Slot 1	0x0010~0x0016
Slot 2	0x0017~0x001D
...	...
Slot 10	0x004F~0x0055

5.4 屏蔽区定义块

- 起始地址: 0x0080
- 每个屏蔽区占 7 个寄存器
- 最多 10 个屏蔽区

结构与感应区完全一致:

偏移	字段	类型	说明
+0	zone_id	uint16	范围 1~10
+1	x_min_mm	int16	单位 mm
+2	x_max_mm	int16	单位 mm
+3	y_min_mm	int16	单位 mm
+4	y_max_mm	int16	单位 mm
+5	z_min_mm	uint16	单位 mm
+6	z_max_mm	uint16	单位 mm

地址展开示例:

屏蔽区槽位	地址范围
Slot 1	0x0080~0x0086
Slot 2	0x0087~0x008D
...	...
Slot 10	0x00BF~0x00C5

5.5 apply_command

主站用 0x06 向 0x00F0 写入命令值:

值	含义
0x0001	应用全局参数
0x0002	应用总体检测边界
0x0003	应用感应区配置
0x0004	应用屏蔽区配置
0x00FF	应用全部暂存配置

5.6 apply_result

主站可通过 0x03 读取 0x00F1 判断最近一次应用结果:

值	含义
0x0000	成功
0x0001	全局参数非法
0x0002	总体检测边界非法
0x0003	感应区配置非法
0x0004	屏蔽区配置非法
0x0005	应用到设备 / 雷达失败

5.7 配置合法性要求

全局参数

参数	范围
motion_sensitivity	0~10
presence_sensitivity	0~10
target_disappear_delay_s	>=2

总体检测边界

1. 必须满足 $\text{detect_x_min_mm} < \text{detect_x_max_mm}$
2. 必须满足 $\text{detect_y_min_mm} < \text{detect_y_max_mm}$
3. 必须满足 $\text{detect_z_min_mm} \leq \text{detect_z_max_mm}$

感应区 / 屏蔽区

1. zone_id 必须在 1~10
2. 同一类区列表内 zone_id 不能重复
3. 必须满足 $x_{\min} < x_{\max}$
4. 必须满足 $y_{\min} < y_{\max}$
5. 必须满足 $z_{\min} < z_{\max}$
6. **允许不同感应区互相重叠**
7. **允许感应区与屏蔽区按各自规则独立配置**

6. Input Registers

6.1 分区结果摘要

地址	名称	类型	说明
0x1000	update_seq_lo	uint16	雷达结果更新序号低 16 位
0x1001	update_seq_hi	uint16	雷达结果更新序号高 16 位
0x1002	active_zone_bitmap	uint16	已启用感应区位图, bit0 对应 zone1
0x1003	occupied_zone_bitmap	uint16	当前有目标的感应区位图, bit0 对应 zone1

位图约定:

- bit0 -> zone1
- bit1 -> zone2
- ...
- bit9 -> zone10

说明:

1. active_zone_bitmap 根据当前生效的感应区配置生成
2. occupied_zone_bitmap 根据当前雷达结果中的 zone_id 生成
3. **不定义总目标数量**

6.2 感应区结果块

每个感应区分配一个固定基地址, 便于 PLC / 上位机直观映射:

感应区 ID	基地址
Zone 1	0x1100
Zone 2	0x1200
Zone 3	0x1300
Zone 4	0x1400

感应区 ID	基地址
Zone 5	0x1500
Zone 6	0x1600
Zone 7	0x1700
Zone 8	0x1800
Zone 9	0x1900
Zone 10	0x1A00

每个感应区结果块内部格式完全一致。

6.2.1 区块头

偏移	名称	类型	说明
+0x00	zone_id	uint16	当前区号
+0x01	zone_target_count	uint16	当前该区目标数

6.2.2 目标槽位

每个目标占 4 个寄存器：

偏移	字段	类型	说明
+0	target_id	uint16	目标 ID
+1	x_mm	int16	单位 mm
+2	y_mm	int16	单位 mm
+3	z_mm	int16	单位 mm

块内最多预留 20 个目标槽位：

目标序号	地址偏移范围
Target 1	+0x02~+0x05
Target 2	+0x06~+0x09
...	...
Target 20	+0x4E~+0x51

说明：

1. 每个区块总共实际使用 0x52 个寄存器
2. 超出当前 zone_target_count 的目标槽位返回 0
3. 不同感应区结果之间不做去重

7. 主站典型访问流程

7.1 读取分区结果

1. 先读 0x1000~0x1003
2. 如果 update_seq 变化, 再根据 occupied_zone_bitmap 读取对应区块
3. 例如读取感应区 2 的结果: 0x1200~0x1251

7.2 修改全局参数

1. 0x06 写 0x0001 运动灵敏度
2. 0x06 写 0x0002 存在灵敏度
3. 0x06 写 0x0003 目标消失延时
4. 0x06 写 0x00F0 = 0x0001
5. 0x03 读 0x00F1 检查应用结果

7.3 修改总体检测边界

1. 0x06 / 0x10 写 0x0004~0x000A
2. 0x06 写 0x00F0 = 0x0002
3. 0x03 读 0x00F1

7.4 修改感应区

1. 0x10 写 0x000B / 0x0010~0x0055
2. 0x06 写 0x00F0 = 0x0003
3. 0x03 读 0x00F1

7.5 修改屏蔽区

1. 0x10 写 0x000C / 0x0080~0x00C5
2. 0x06 写 0x00F0 = 0x0004
3. 0x03 读 0x00F1

8. 协议模型摘要

EDV21C-4 的 Modbus RTU 对外模型是：

- **全局参数：**运动灵敏度、存在灵敏度、目标消失延时
- **总体检测边界：**安装高度、检测范围 XY、检测高度
- **区域配置：**感应区 / 屏蔽区列表
- **运行结果：**按 zone_id 分块输出人员坐标

协议 **不提供** 全局人数或全局目标总数，因为重叠感应区会让同一目标在多个区块中重复出现。

9. 通过 UART 直连发送指令的测试示例

以下示例均假设：

1. 从站地址为 1
2. 串口参数为 9600 8N1
3. 发送内容为 **完整 Modbus RTU 帧**，末尾已包含 CRC16
4. 可使用串口调试助手的“HEX 发送”模式直接下发
5. **拨码开关 1 = OFF**，设备工作在 UART 直连模式

9.1 读取 0x0000~0x000A

用途：读取协议版本、全局参数、总体检测边界。

- 请求帧：

01 03 00 00 00 0B 04 0D

- 含义：
 - 01: 从站地址
 - 03: 读 Holding Registers
 - 00 00: 起始地址 0x0000
 - 00 0B: 读取 11 个寄存器
- 正常响应格式：

01 03 16 [22 bytes payload] [CRC_L CRC_H]

9.2 读取输入摘要区 0x1000~0x1003

用途: 读取 update_seq、active_zone_bitmap、occupied_zone_bitmap。

- 请求帧:

01 04 10 00 00 04 F5 09

- 正常响应格式:

01 04 08 [8 bytes payload] [CRC_L CRC_H]

9.3 读取感应区 1 结果块

用途: 读取 zone_id=1 的结果块 0x1100~0x1151。

- 请求帧:

01 04 11 00 00 52 74 CB

- 说明:
 - 0x1100: Zone 1 基地址
 - 0x0052: 共读 82 个寄存器

9.4 读取感应区 1 的前 16 个寄存器

用途: 读取 Zone 1 的 zone_id、zone_target_count 以及前几个目标槽位, 适用于联调阶段的结果确认。

- 请求帧:

01 04 11 00 00 10 F4 FA

- 说明:
 - 该指令适用于联调阶段的结果快速确认
 - 若当前区内目标数小于读取槽位数, 多余寄存器返回 0

9.5 写运动灵敏度并应用

1. 写 motion_sensitivity = 5

01 06 00 01 00 05 18 09

2. 写 apply_command = 0x0001, 应用全局参数

01 06 00 F0 00 01 48 39

3. 读取 apply_result

01 03 00 F1 00 01 D5 F9

返回 0x0000 表示应用成功。

9.6 写存在灵敏度并应用

1. 写 presence_sensitivity = 6

01 06 00 02 00 06 68 08

2. 写 apply_command = 0x0001

01 06 00 F0 00 01 48 39

3. 读取 apply_result

01 03 00 F1 00 01 D5 F9

9.7 写总体检测边界并应用

示例值:

- install_height_mm = 3000
- detect_x_min_mm = -3000
- detect_x_max_mm = 3000
- detect_y_min_mm = 0
- detect_y_max_mm = 5000
- detect_z_min_mm = 0
- detect_z_max_mm = 3000

1. 用 0x10 写 0x0004~0x000A

01 10 00 04 00 07 0E 0B B8 F4 48 0B B8 00 00 13 88 00 00 0B B8 E4 9F

2. 应用总体检测边界

01 06 00 F0 00 02 08 38

3. 读取 apply_result

01 03 00 F1 00 01 D5 F9

9.8 写感应区数量并应用

以下示例将感应区数量写为 2，随后应用感应区配置:

1. 写 sensing_zone_count = 2

01 06 00 0B 00 02 F9 C8

2. 写 apply_command = 0x0003

01 06 00 F0 00 03 C9 F8

3. 读取 apply_result

01 03 00 F1 00 01 D5 F9

9.9 Modbus 异常响应判断

若设备返回异常帧，格式如下:

01 83 02 C0 F1

其中:

- 0x83: 0x03 | 0x80, 表示功能码异常响应
- 0x02: 非法寄存器地址

常见异常码:

异常码	含义
0x01	非法功能码
0x02	非法寄存器地址
0x03	非法寄存器数量 / 非法参数值

10. Modbus RTU 主机测试脚本

仓库已提供命令行上位机脚本:

tools\modbus_rtu_host.py

依赖:

pip install -r .\tools\ota_updater\requirements.txt

10.1 查看帮助

```
python .\tools\modbus_rtu_host.py --help
```

10.2 查看示例帧

```
python .\tools\modbus_rtu_host.py examples
```

10.3 列出串口

```
python .\tools\modbus_rtu_host.py list-ports
```

10.3.1 使用前的拨码说明

1. **RS485 调试**: 拨码开关 1 = ON, 串口工具 / USB 转 485 工具接 A/B
2. **UART 直连调试**: 拨码开关 1 = OFF, A 连接外部系统 RX, B 连接外部系统 TX, 并与外部系统共地
3. 两种模式下脚本命令保持一致, 仅连接方式不同

10.4 读取 Holding Registers

```
python .\tools\modbus_rtu_host.py read-holding --port COM6 --slave 1 --start 0x0000 --count 0x000B
```

10.5 读取 Input Registers

```
python .\tools\modbus_rtu_host.py read-input --port COM6 --slave 1 --start 0x1000 --count 0x0004
```

10.6 读取感应区 1 结果块

```
python .\tools\modbus_rtu_host.py read-input --port COM6 --slave 1 --start 0x1100 --count 0x0052
```

读取 Zone 1 前 16 个寄存器的示例如下:

```
python .\tools\modbus_rtu_host.py read-input --port COM6 --slave 1 --start 0x1100 --count 0x0010
```

10.7 写单个寄存器

```
python .\tools\modbus_rtu_host.py write-single --port COM6 --slave 1 --register 0x0001 --value 5  
python .\tools\modbus_rtu_host.py write-single --port COM6 --slave 1 --register 0x00F0 --value 0x0001
```

写存在灵敏度并应用的示例如下:

```
python .\tools\modbus_rtu_host.py write-single --port COM6 --slave 1 --register 0x0002 --value 6  
python .\tools\modbus_rtu_host.py write-single --port COM6 --slave 1 --register 0x00F0 --value 0x0001  
python .\tools\modbus_rtu_host.py read-holding --port COM6 --slave 1 --start 0x00F1 --count 0x0001
```

10.8 写多个寄存器

下面示例一次写入总体检测边界:

```
python .\tools\modbus_rtu_host.py write-multiple --port COM6 --slave 1 --start 0x0004 3000 -3000 3000 0 50  
python .\tools\modbus_rtu_host.py write-single --port COM6 --slave 1 --register 0x00F0 --value 0x0002
```

10.9 发送原始 RTU 帧

```
python .\tools\modbus_rtu_host.py raw --port COM6 --hex "01 03 00 00 00 0B"
```

默认会自动补 CRC; 如果输入内容已经带 CRC, 可使用:

```
python .\tools\modbus_rtu_host.py raw --port COM6 --hex "01 03 00 00 00 0B 04 0D" --no-crc
```

10.10 UART 直连查询 / 设置示例

拨码开关 1 = OFF 时, 可参照以下命令进行 UART 直连查询与设置测试:

```
python .\tools\modbus_rtu_host.py read-input --port COM6 --slave 1 --start 0x1000 --count 0x0004
python .\tools\modbus_rtu_host.py read-input --port COM6 --slave 1 --start 0x1100 --count 0x0010
python .\tools\modbus_rtu_host.py write-single --port COM6 --slave 1 --register 0x0001 --value 5
python .\tools\modbus_rtu_host.py write-single --port COM6 --slave 1 --register 0x00F0 --value 0x0001
python .\tools\modbus_rtu_host.py read-holding --port COM6 --slave 1 --start 0x00F1 --count 0x0001
```